

Technical Documentation

Small Graphics Engine

Ivan Džanija

`ivan.dzanija@fer.unizg.hr`

University of Zagreb, Faculty of Electrical Engineering and Computing

June 7, 2026

Version 1.0

Contents

1	Introduction	2
2	Drawing lines on the screen	2
3	Drawing and shading polygons	3
4	Starting with OpenGL	5
5	3D Models	6
6	Math and transformations	8
7	Back-face culling	9

1 Introduction

This is the technical documentation for the Small Graphics Engine project developed for the laboratory exercise in the Interactive Computer Graphics course at the University of Zagreb. The purpose of this document is to provide an overview of the system architecture, design decisions, and implementation details.

2 Drawing lines on the screen

This section describes the algorithm used for drawing lines on the raster display. The default algorithm for doing such a task is Bresenham's line algorithm. That is also the algorithm implemented here, although only for education purposes, later on the line drawing is done using OpenGL's built-in line drawing capabilities that fully utilize the GPU's capabilities for rasterization. Bresenham's line algorithm here is implemented fully on the CPU, with manual fragment shading. The screen raster is kept in memory as a 2D array of pixels. Here is the code snippet for the manual fragment shading:

```

1 // Raster is kept as
2
3 // Shades a fragment at the given (x, y) coordinates with the specified
  color.
4 int32 Graphics::shade_fragment(int32 x, int32 y, glm::vec3 color) {
5     y = _height - 1 - y; // Invert y-axis
6     if (x >= 0 && x < _width && y >= 0 && y < _height) {
7         _raster(x, y) = color;
8         // _raster[(y * _width * 3) + (x * 3)] = color.x;
9         // _raster[(y * _width * 3) + (x * 3) + 1] = color.y;
10        // _raster[(y * _width * 3) + (x * 3) + 2] = color.z;
11        return 0;
12    }
13    std::cerr << "ERROR: fragment out of bounds: (" << x << ", " << y <<
        ")" << std::endl;
14    return -1;
15 }
```

Listing 1: Manual fragment shading example

While internal representation of the raster is defined as:

```

1 struct Raster {
2     int32 width{};
3     int32 height{};
4     std::vector<glm::vec3> data;
5 };
```

Listing 2: Raster representation example

As for the line drawing algorithm itself more information (such as full derivation) can be found [here](#). The snippet of the line drawing algorithm is as follows:

```

1 // 0 to 90 degrees
2 void bresenham_line_1(eng::Graphics &screen, int32 x0, int32 y0, int32
    x1, int32 y1,
3                     const glm::vec3 &color) {
4     const int32 dy = y1 - y0;
5     const int32 dx = x1 - x0;
6     if (dy <= dx) {
7         const int32 change = dy << 1; // 2 * dy
```

```

8     const int32 correction = -dx << 1;    // -2 * dx
9
10    int32 y_curr = y0;
11    int32 y_cal = -dx;
12    for (int32 x_curr = x0; x_curr <= x1; ++x_curr) {
13        screen.shade_fragment(x_curr, y_curr, color);
14        y_cal += change;
15        if (y_cal >= 0) {
16            ++y_curr;
17            y_cal += correction;
18        }
19    }
20 } else {
21     const int32 change = dx << 1;          // 2 * dx
22     const int32 correction = -dy << 1;    // -2 * dx
23
24     int32 x_curr = x0;
25     int32 x_cal = -dy;
26     for (int32 y_curr = y0; y_curr <= y1; ++y_curr) {
27         screen.shade_fragment(x_curr, y_curr, color);
28         x_cal += change;
29         if (x_cal >= 0) {
30             ++x_curr;
31             x_cal += correction;
32         }
33     }
34 }
35 }

```

Listing 3: Bresenham's line algorithm example

3 Drawing and shading polygons

In this section we describe the algorithm for drawing and shading polygons. To be more precise we are talking about convex polygons, as the algorithm for concave polygons is more complex and is not implemented here. In this section we are still doing the drawing and shading on the CPU, with manual fragment shading as this is this educational implementation and most of the Graphics API's (such as OpenGL) provide built-in functionality for drawing and shading polygons that fully utilize the GPU's capabilities. First thing we are doing is providing a user with a way to define a polygon. When the user presses the left mouse button the application records the position of the mouse cursor and adds it to the list of polygon vertices. If the vertex just added would form a non-convex polygon the user is alerted in the console. The checking is done by seeing if all the vertices of the polygon are on the same side of the line formed by any two vertices of the polygon. If that is not the case then the polygon is not convex. Here is the code snippet for checking if a polygon is convex:

```

1 [[nodiscard]] std::optional<bool> test_convex() const {
2     if (_elements.size() < 3) {
3         return std::nullopt;    // Not enough vertices to determine convexity
4     }
5     uint8 negative = 0;
6     uint8 positive = 0;
7     for (size_t i = 0; i < _elements.size(); ++i) {
8         const auto &curr_edge =
9             _elements[(i + _elements.size() - 1) % _elements.size()].edge;
10        const auto &next_point = _elements[(i + 1) % _elements.size()].
            point;

```

```

11     double dot =
12         (curr_edge.a * next_point.x) + (curr_edge.b * next_point.y) +
13         curr_edge.c;
14     if (dot < 0) {
15         ++negative;
16     }
17     if (dot > 0) {
18         ++positive;
19     }
20     return (negative == 0 || positive == 0);
21 }

```

Listing 4: Convex polygon checking example

The the user finishes defining the polygon by pressing the right mouse button the last line between the last vertex and the first vertex is drawn. After that the polygon is filled using the scanline algorithm. The scanline algorithm works by iterating through each scanline (horizontal line) of the raster and finding the intersection points between the scanline and the left and right edges of the polygon. Specifically for each scanline we are looking for the leftmost right edge and the rightmost left edge. After that we are filling the pixels between those two edges using the Bresenham's line algorithm described in the previous section. The code snippet for the scanline algorithm is as follows:

```

1 void draw_filled(eng::Graphics &screen) {
2     if (_elements.empty()) {
3         // std::cerr << "Polygon has no vertices to draw." << std::endl;
4         return; // No vertices to draw
5     }
6
7     int32 x_min = _elements[0].point.x;
8     int32 x_max = _elements[0].point.x;
9     int32 y_min = _elements[0].point.y;
10    int32 y_max = _elements[0].point.y;
11    double L;
12    double R;
13
14    for (const auto &element : _elements) {
15        x_min = std::min(x_min, element.point.x);
16        x_max = std::max(x_max, element.point.x);
17        y_min = std::min(y_min, element.point.y);
18        y_max = std::max(y_max, element.point.y);
19    }
20
21    for (int32 y = y_min; y <= y_max; ++y) {
22        L = static_cast<double>(x_max);
23        R = static_cast<double>(x_min);
24        for (const auto &curr : _elements) {
25            if (curr.edge.a == 0) {
26                continue; // Skip horizontal edges
27            }
28            double x = (-curr.edge.b * y - curr.edge.c) / static_cast<double>
29                >(curr.edge.a);
30            if (test_orientation() == PolyOrientation::CLOCKWISE) {
31                if (curr.left) {
32                    L = std::min(L, x);
33                } else {
34                    R = std::max(R, x);
35                }
36            }
37        }
38        // Fill the scanline from L to R
39    }
40 }

```

```

34     }
35   } else {
36     if (curr.left) {
37       R = std::max(R, x);
38     } else {
39       L = std::min(L, x);
40     }
41   }
42 }
43 eng::draw_line(screen, static_cast<int32>(L + 0.5F), y,
44                  static_cast<int32>(std::round(R + 0.5F)), y);
45 }
46 // Draw starting vertex in red and outline in blue for clarity
47 screen.shade_fragment(_elements[0].point.x, _elements[0].point.y,
48                      glm::vec3(1, 0, 0));
49 draw_outline(screen);
50 }

```

Listing 5: Scanline algorithm example

The final thing supported by the application is testing if the point is inside the polygon. This is done by counting the number of times a ray from the point intersects the edges of the polygon. The user defines a point by pressing the left mouse button while in polygon testing mode. If the number of the intersections is odd then the point is inside the polygon, otherwise it is outside. Example are shown in the picture [1](#).

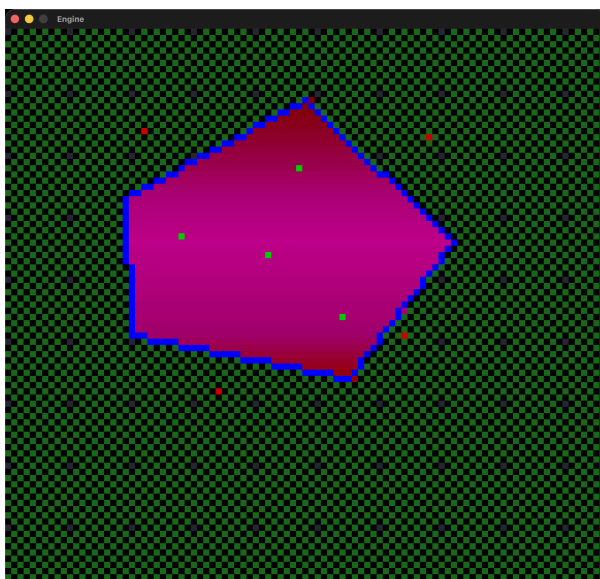


Figure 1: Example of point-in-polygon testing. The red point is outside the polygon, while the green point is inside.

4 Starting with OpenGL

Until now we have been doing all the drawing and shading on the CPU, with manual fragment shading and just forwarding the final raster to the GPU for displaying. However, this is highly inefficient and we are now switching to using OpenGL's built-in functionality to fully utilize the GPU's capabilities for rasterization. In this part we will be defining and operating with OpenGL's pipeline. In the following picture [2](#) you can see the final result of the current stage of

the application. The top left corner is just the display for the currently selected color. While the triangles are drawn using OpenGL's built-in triangle primitives. The triangles are colored using the color defined in the top left corner and the color is interpolated across the triangle. All of this is done using OpenGL's built-in functionality and the GPU's capabilities. Only thing we do here is record the position of the mouse cursor when the user clicks and update the corresponding vertex buffer.

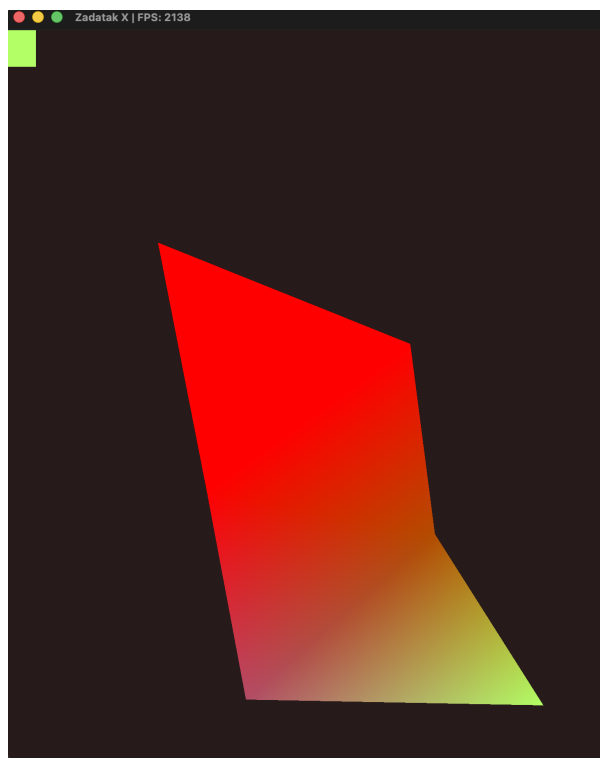


Figure 2: Example of basic triangle coloring using OpenGL's built-in functionality.

OpenGL's features we introduced here are vertex and fragment shaders, vertex buffer objects (VBOs), vertex array objects (VAOs) and element buffer objects (EBOs). Vertex and fragment shaders we used here are the most basic ones that just pass the vertex positions and colors to the GPU and let the GPU handle the rasterization and interpolation. The VBOs are used to store the vertex data (positions and colors) on the GPU, while the VAOs are used to store the state of the vertex attributes (such as the layout of the vertex data). The EBOs are used to store the indices of the vertices when we going with indexed approach.

5 3D Models

Following the section where we introduced OpenGL into our application we now want to jump from 2D space to 3D space and start working on 3D models. First thing we need to do is define a good architecture for the rest of the application since we will be using this as a base for the rest of the development. The beginning idea for the architecture is given in the following picture 3. Although some small changes were made later on we will keep this here since for now this was a plan and the mental model. After we have the architecture defined we start building the pieces. First thing we are doing is defining a way to load 3D models from files. We are using the OBJ file format for storing 3D models and using external library Assimp for loading the models from files. All models are represented by the class `Renderable` that is just an abstract interface for rendering something on the screen. In this section we are focusing on loading models and for

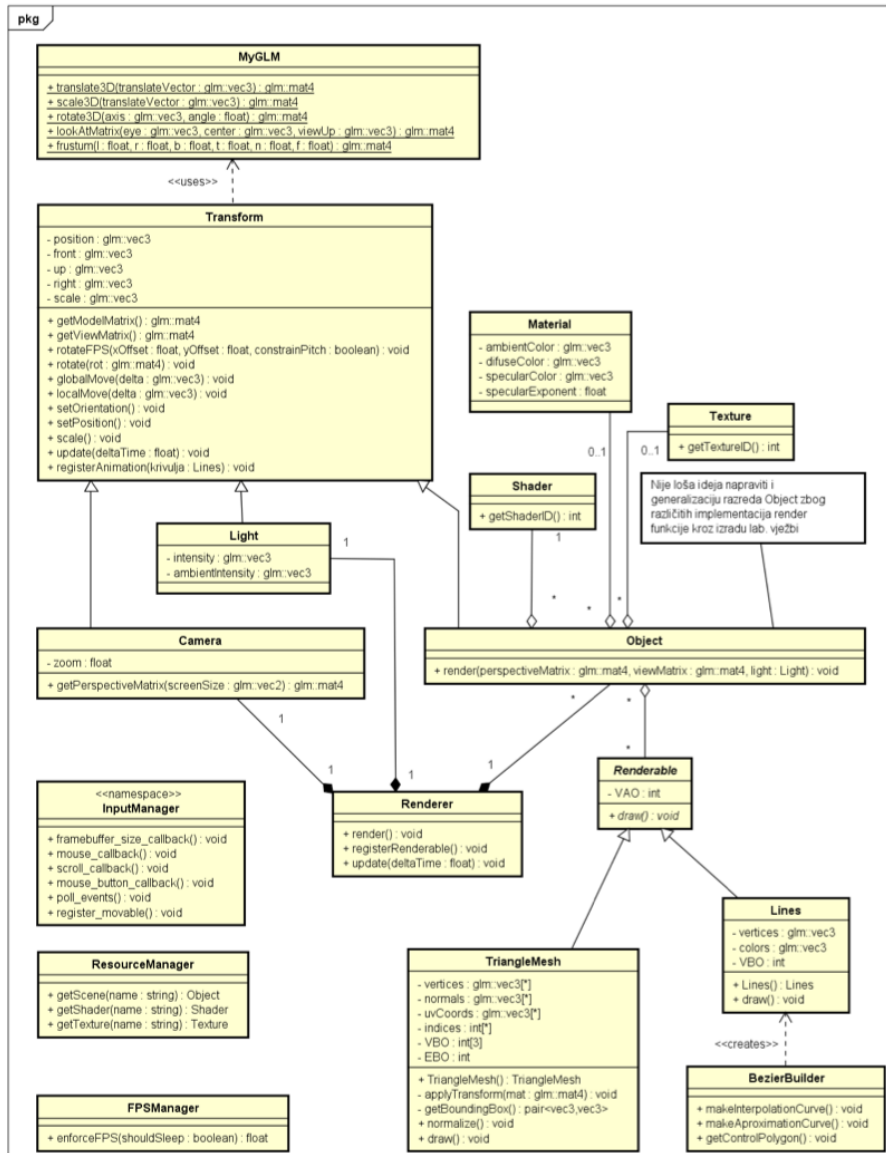


Figure 3: Architecture of the application.

that we introduce another class **Mesh** that is responsible for storing the vertex data (positions, normals, texture coordinates) and the indices of the vertices. The **Renderable** class has a method **draw()** that is responsible for rendering the model on the screen. Other pieces we introduce here are **Object** class and **Renderer** class. We will start with the **Object** class and it is responsible for storing the shader and all the meshes that make up the model. The **Renderer** class is responsible for rendering all the objects on the screen and it keeps track of all the objects in the scene with methods for adding and removing objects from the scene. The final things we introduce are the **ResourceManager** class and the **ObjectLoader** class. The **ResourceManager** is the owner off all the objects and resources in the application and it caches everything inside appropriate data structures like `std::unordered_map` for easy access. When some resource is needed somewhere in the application it is requested from the **ResourceManager**. From there there are two options, either the resource is already cached and we return a `std::shared_ptr` to the resource or the resource is not cached then we use the other mentioned class **ObjectLoader** that is just a clean facade pattern for using the Assimp library for loading the models from files. After the model is

loaded it is first normalized on the screen and then cached in the ResourceManager. After the model is loaded and cached then we return a `std::shared_ptr` to the model and it is used in the application.

6 Math and transformations

This is probably the hardest part of the application when doing manually. In this section we introduce the math library GLM, classes like Transform and TransformGenerator and the concept of camera. We will start with the Transform class. The idea of the Transform class is to provide a clean interface for everything that will be in the scene and needs to track its position, rotation and scale. The Transform class is abstract class from which Camera and Object classes inherit. Transform class internally keeps track of the position, local axes and scale of the object. It provides methods for rotating, moving and scaling the object in the scene. These methods all just internally update the position, local axes and scale. This movement is then represented as a transformation matrix that the application fetches with method `get_model_matrix()` and sends the model matrix to the shader for rendering. Another class we introduce is the TransformGenerator class. This class is just a clean facade pattern for generating transformation matrices using the GLM library. The final thing introduced here is the concept of camera encapsulated in the Camera class. The Camera class inherits from the Transform class and only thing it is responsible for is generating the view and projection matrices. The view matrix is generated using `glm::lookAt()` function from the GLM library and the projection matrix is generated using `glm::frustum()` function from the GLM library (rather the abstracted versions inside the TransformGenerator class). With all this we finally arrive at the shaders. Fragment shader has stayed the same and we are not making changes now. The vertex shader has a small change. Now we are sending the model, view and projection matrices to the vertex shader and then all the transformations are done in the vertex shader (meaning on the GPU). The vertex shader looks as follows:

```

1 #version 330 core
2 // Input
3 layout (location = 0) in vec3 aPos;
4 layout (location = 1) in vec3 aCol;
5
6 // Matrix
7 uniform mat4 u_model;
8 uniform mat4 u_view;
9 uniform mat4 u_projection;
10
11 // Color
12 uniform vec3 u_color;
13
14 // Output
15 out vec3 color;
16
17 void main() {
18     color = aCol;
19     gl_Position = u_projection * u_view * u_model * vec4(aPos, 1.0);
20 }
```

Listing 6: Vertex shader example

After all this we finally have a working application that can load 3D models from files, render them and move around the scene. The preview of the application at this stage is shown in the following picture 4. In the picture we can also see the local axes of the models and global axes of the scene.

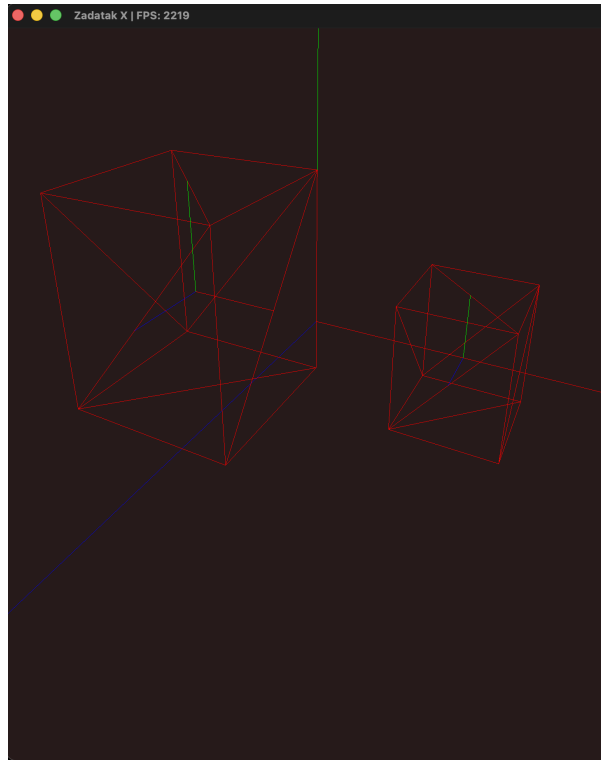


Figure 4: Preview of the application at the stage of 3D model loading and rendering.

7 Back-face culling

This is a small optimization technique that we can implement to improve the performance of our application. The idea is to not render any faces of the model that are facing away from the camera. This can be done a lot of different ways. We have tried two approaches for educational purposes. Both approaches are done inside the geometry shader. The first approach is done in the scene coordinate system where we calculate the *centroid* of the triangle and then calculate the normal of the triangle. After that we calculate the vector from the camera to the centroid and then we calculate the dot product between the normal and the vector from the camera to the centroid. The dot product gives us the angle between the normal and the vector from the camera to the centroid. If the angle is greater than 90 degrees then the face is facing away from the camera and we can discard it.